INTERVIEW QUESTIONS

SV & UVM

VLSIFlow.com

1. GENERAL – QUESTIONS FROM UVM & SV

1. What are assertions? Types of assertions?

• Assertions are used to validate the behaviour of a system defined as properties
& Also used in functional coverage. If a property of the design that is being checked for by an assertion doesn't behave in the expected way, assertion fails.
Ex. Assume the design requests grant & expects to receive an ack within 4 clock cycles. But the design gets an ack at 5^{th} clock cycle then assertion fails.

There are 2 types of assertion:
1. Immediate Assertions
2. Concurrent Assertions

1. **Immediate Assertions** – are executed like a statement in a procedural block & follow simulation event semantics. These are used to verify immediate property during simulation
2. **Concurrent Assertions** – are based on clock semantics. Properties get evaluated everytime on given clock.

2. What is functional coverage? How is it different from code coverage?

- **Functional coverage** is a measure of how functionalities/featuresofthe design have been exercisedby thetests. Thisisuseful in constrained randomverification (CRV) to know what features have been covered by a set of tests in regression.

Code coverage measures how much of the design code is exercised. This includes the execution of design blocks, Number of Lines, Conditions, FSM, Toggle and Path.

3.What is uvm_config_db ? What is difference between uvm_config_db & uvm_resource_db?

- uvm_config_db is a parameterized class used for configuration of different type of parameter into the uvm database, so that it can be used by any component in the lower level of hierarchy.

- uvm_config_db is a convenience layer built on top of uvm_resource_db, but that convenience is very important. In particular, uvm_resource_db uses a "last write wins" approach. The uvm_config_db, on the other hand, looks at where things are in the hierarchy up through end_of_elaboration, so "parent wins." Once you start start_of_simulation, the config_db becomes "last write wins."

4. What is uvm RAL model? why it is required?

- In a verification context, a register model(orregister abstraction layer) is a set of classes that model the memory mapped behaviorofregisters and memories in the DUT in order to facilitate stimulus generation and functional checking (and optionally some aspects of functional coverage). The UVM providesaset of base classes that can be extended to implement comprehensive register modelingcapabilities.

5. What is the difference between new () and create?

- We all know about new () method that is use toallocate memory to an object instance. In UVM (and OVM), the create () method causesanobject instance to be created from the factory. This allows you to use factory overridestoreplace the desired object with an object of a different type without having to recode.

6.What is analysis port?

- Analysis port (class uvm_tlm_analysis_port) — a specific type of transaction-level port that can be connected to zero, one, or many analysis exports and through which a component may call the method write implemented in another component, specifically a subscriber.

- port, export, and impclasses used for transaction analysis.

- **uvm_analysis_port**
  Broadcasts a valuetoall subscribers implementing a uvm_analysis_imp.
- **uvm_analysis_imp**

- Receives all transactions broadcasted by a uvm_analysis_port.

  uvm_analysis_export

  Exports a lower-leveluvm_analysis_imp to its parent.

7. What is TLM FIFO?

- In simpler wordsTLM FIFO is a FIFO between two UVM components, preferably between Monitor and Scoreboard. Monitor keep on sending the DATA, which will be stored in TLM FIFO, and Scoreboard can get data from TLM FIFO whenever needed.

8. What is the difference between UVM RAL model backdoor write/read and front door write/read?

- Fontdoor accessmeans using the standard access mechanism external to the DUT to read or write to aregister.This usually involves sequences of time-consuming transactions on a bus interface.

- Backdooraccessmeans accessing a register directly via hierarchical reference or outside the languageviathePLI. A backdoor reference usually in 0 simulation time.

9. What is objection?

- The objectionmechanism in UVM is to allow hierarchical status communication among componentswhichishelpful in deciding the end of test.

- There is abuilt-inobjection for each in-built phase, which provides a way for components and objectstosynchronize their testing activity and indicate when it is safe to end the phase and, ultimately,thetest end.

- The component or sequence will raise a phase objection at the beginning of an activity that must be completed before the phase stops, so the objection will be dropped at the end of that activity. Once all of the raised objections are dropped, the phase terminates. Raising an objection: phase.raise_objection(this); Dropping an objection: phase.drop_objection(this);

10. What is p_sequencer ? OR Difference between m_sequencer and p_sequencer?

- m_sequencer is the default handle for uvm_vitual_sequencer and p_sequencer isthe hook up for child sequencer.

- **m_sequencer** is the generic uvm_sequencer pointer. It will always exist for the uvm_sequence and is initialized when the sequence is started.

- **p_sequencer** is a typed-specific sequencer pointer, created by registering the sequence to the sequencer using macros (`uvm_declare_p_sequencer) . Being type specific, you will be able to access anything added to the sequencer (i.e., pointers to other sequencers, etc.). p_sequencer will not exist if we have not registered the sequence with the `uvm_declare_p_sequencer macros.

- The drawback of p_sequencer is that once the p_sequencer is defined, one cannot run the sequence on any other sequencer type.

11. What is the difference between Active mode and Passive mode with respect to agent?

- An agent is a collection of a sequencer, a driver and a monitor.
- In **active mode**, the sequencer and the driver are constructed and stimulus is generated by sequences sending sequence items to the driver through the sequencer. At the same time the monitor assembles pin level activity into analysis transactions.
- In **passive mode**, only the monitor is constructed and it performs the same function as in an active agent. Therefore, your passive agent has no need for a sequencer. You can set up the monitor using a configuration object.

12. What is the difference betweencopyandclone?

- Thebuilt-in **copy ()** methodexecutesthe__m_uvm_field_automation() method with the requiredcopycodeasdefinedbythefieldmacros(ifused)and then calls the built-in **do_copy()** virtualfunction.Thebuilt-indo_copy()virtualfunction, as defined in the uvm_object baseclass,isalsoanemptymethod,soiffieldmacros are used to define the fields of the transaction,thebuilt-incopy()methodwillbepopulated with the proper code to copy the transactionfieldsfromthefieldmacrodefinitionsand then it will execute the empty do_copy() method,whichwillperformnoadditionalactivity.
- The **copy ()** method canbeusedasneededintheUVMtestbench. One common place where the copy () methodisusedistocopythesampledtransaction and pass it into a sb_calc_exp() (scoreboardcalculateexpected)externalfunctionthat is frequently used by the scoreboard predictor.
- The **clone ()** methodcallsthecreate()method(constructsanobject of the same type) and then calls the copy()method.Itisaone-stepcommandtocreate and copy an existing object to a new objecthandle.

13. What is UVM factory?

- UVM Factory is usedto manufacture (create) UVM objects and components. Apart from creating the UVMobjects and components the factory concept essentially means that you can modify or substitute the nature of the components created by the factory without making changes tothetestbench.
- For example, if youhave written two driver classes, and the environment uses only one of them. By registeringboth the drivers with the factory, you can ask the factory to substitute the existing driverinenvironment with the other type. The code needed to achieve this is minimal, and canbewritten in the test.

14. What are the types of sequencers? Explain each?

- There are two types of sequencers:

- **uvm_sequencer # (REQ, RSP):**

When the driver initiates new requestsforsequences, the sequencer selects a sequence from a list of available sequences to produceanddeliverthe next item to execute. In order to do this, this type of sequencer is usually connectedtoadriver uvm_driver # (REQ, RSP).

- **uvm_push_sequencer # (REQ, RSP):**

The sequencer pushes new sequence itemstothedriver, butthedriver has the ability to block the item flow when it's not ready to acceptanynew transactions. This type of sequencer is connected to a driver of type uvm_push_driver#(REQ, RSP).

15. What is virtual sequencer and virtual sequence in UVM?

- A virtual sequencer is a sequencer that is not connectedtoa driver itself, but contains handles for sequencers in the testbench hierarchy. It is anoptional component for running of virtual sequences - optional because they need no driverhookup, instead calling other sequences which run on real sequencers.

- A sequence which controls stimulus generation across more than one sequencer, coordinate the stimulus across different interfaces and the interactions between them. Usually, the top level of the sequence hierarchy i.e., 'master sequence' or 'coordinator sequence'. Virtual sequences do not need their own sequencer, as they do not link directly to drivers. When they have one it is called a virtual sequencer.

16. How set_config_* works?

- The *uvm_config_db* class provides a convenience interface on top of the uvm_resource_db to simplify the basic interface that is used for configuring uvm_component instances.

- Configuration is a mechanism in UVM that higher level components in a hierarchy can configure the lower-level components variables. Using ***set_config_ **** methods, user can configure integer, string and objects of lower-level components. Without this mechanism, user should access the lower-level component using hierarchy paths, which restricts re-usability.

- This mechanism can be used only with components. Sequences and transactions cannot be configured using this mechanism. When ***set_config_ **** method is called, the data is stored w.r.t strings in a table. There is also a global configuration table.

- Higher level component can set the configuration data in level component table. It is the responsibility of the lower-level component to get the data from the component table and update the appropriate table.

• following are the method to configure integer, string and object of uvm_object based class respectively.

function void **set_config_int** (string inst_name, string field_name, uvm_bitstream_t value)
function void **set_config_string** (string inst_name, string field_name, string value)
function void **set_config_object** (string inst_name, string field_name, uvm_object value, bit clone = 1)

17. What are the advantages of uvm RAL model?

- TheRAL(registerabstractionlayer)providesaccessestoDUTandalsokeepsatrackof register content of DUT.
- UVMRALcanbeusedtoautomatethecreationofhigh-level,object-orientedabstraction model of registers and memory in DUT.
- Registerlayermakestheregisterabstractionandaccessofitscontentsindependentofthe bus protocolwhichis usedtotransfer datainandout ofregisters insidethe design.
- HierarchicalmodelprovidedbyRALmakesthereusabilityoftestbenchcomponentsvery easy.
- The easilychanges in initial configuration of registers or specifications can be communicatedintheentireenvironment.RALlayersupportsbothfrontdoorandbackdoor access.Thebackdooraccessdoesnotusethebusinterfaceratherituses theHDLdefined pathsfordirectcommunicationwiththedevice.Thus,inzerosimulationtimetheregisters of devicecanbereconfiguredusingthebackdooraccessandverificationcanbestarted.
- Onemoreadvantageofbackdooraccessisthatitcanbeusedforverifyiftheaccess throughfrontdoorishappeningcorrectly.Toachievethisthefrontdoor,writeisverified usingbackdoor read.

18. ExplainendofsimulationinUVM?

- Different approachestofinishthe UVM Test usingthe objectionmechanismare 1.
- Raising&droppingobjections raise_objection() and drop_objection() are the methods to be
- used to do that. 2. phase_ready_to_end
- phase_ready_to_endmethodisexecutedautomaticallybyUVMonce‘alldropped’
- conditionisachievedduringRunPhase. 3. set_drain_time AnotherapproachsupportedbyUVMissettingthedraintimeforthesimulation
- environment.Draintimeconceptisrelatedtotheextratimeallocatedtothe UVM
- environmenttoprocesstheleftoveractivitiese.g.lastpacketanalysis&comparisonetc after all thestimulus is applied& processed.

19. What is Virtual interface and how Virtual interface is used?

- **"Virtual interfaces** are class data memberhandles that pointtoan interface instance. This allows a dynamic class object to communicate with a DesignUnder Test (DUT) module instance without using hierarchical references to directly accessthe DUT module ports or internal signals."

1. Difference between bit [7:0] and byte

- Bit is an unsigned 2-state data type that can take only values of either '0' or '1'.
- Byte is a signed variable of 2-state data type that can be used to count values till 127.
- Default value of 2-state data type is '0'.
- A logic [7:0] variable can be used for an unsigned 8-bit variable that can count up to 255.

2. Difference between packed and unpacked array

- A packed array represents a contiguous set of bits and can be made of only the single bit data types (bit, logic, reg) or enumerated types.
- Unpacked array need not be represented as a contiguous set of bits and can be made of any data type.
- In terms of declarations, packed array refers to dimensions declared after the type and before the data identifier name. Bit [7:0] data; //packed array of scalar bit
- Unpacked array refers to the dimensions declared after the data identifier name. Real latency [7:0]; //unpacked array of real

3. Difference between Shallow copy and deep copy

- Shallow copy is the most basic type where only the properties of the class are copied (including handles) but doesn't copy objects. In shallow copy all the properties are copied to new memory location except objects.
- Deep copy first creates a new object for every sub-class to be copied and then copies the value, so every copied sub-class has a different object. In deep copy all the properties and objects are copied to a new memory location.

4. What is meant by virtual

- A virtual class is a class declaration that has the virtual keyword preceding the class keyword. Virtual classes cannot be constructed directly. Any attempt to new ()–construct a virtual class object will give a compilation error. Virtual classes are classes that must be extended in order to use the virtual class functionality. A virtual class could be a top-level base class or an extension of another virtual class. SystemVerilog class methods can be
- either non-virtual or virtual. Virtual methods include the virtual keyword before the function or task keyword. Classes with non-virtual methods fix the method code to the class object when constructed. Virtual method functionality is set at run-time, which allows extended class handles to be assigned to base class handles and run-time method calls using the base class handle will execute the extended class method functionality.

5. What is Polymorphism

- Polymorphism means many forms. Polymorphism in SystemVerilog provides an ability to an object to take on many forms.
- Method handle of super-class/parent-class can be made to refer to the subclass method, this allows polymorphism or different forms of the same method.
- Polymorphism allows the use of a variable of the base class type to hold sub-class objects and to reference the methods of those sub-classess directly from the super-class/parent-class variable.
- It also allows a child class method to have a different definition than its parent class, if parent class method is virtual in nature.

6. What are Constraints

- Constraints are used to restrict the random data generation to meaningful values.
- Writing constraints to a random variable, the user can get specific value on randomization. constraints to a random variable shall be written in constraint blocks.
- Constraint blocks are class members like tasks, functions, and variables
- Constraint blocks will have a unique name within a class
- Constraint blocks consist of conditions or expressions to limit or control the values for a random variable
- Constraint blocks are enclosed within curly braces {}
- Constraint blocks can be defined inside the class or outside the class like extern methods, constraint block defined outside the class is called as extern constraint block

- Set Membership constraint

```
constraint addr_range { addr inside {1,3, [5:10],12, [13:15]};}
```

- Distributive constraint

Weighted distribution weight will be specified to the values inside the constraint block. Value with the more weight will get allocated more often to a random variable.

```
constraint addr_range {addr1 dist {2: = 5, 7: = 8, 10: = 12};}
constraint addr_range {addr2 dist {[ 100: 200]: / 200};}
```

- Conditional constraint

Implication constraints

```
constraint address_range { (addr_range >= 3'd7) -> (addr < 8);}
```

If else constraints

```
constraint address_range {if(addr_range >= 3'd7)
                              addr < 8;
                          else
                              addr > 8;}
```

- Inline constraint

Inline constraint allows the user to add extra constraints to existing constraints written inside the class. inline constraints will be written outside the class i.e along with the randomize method call.

```
initial
 begin
   packet pkt;
   pkt = new ();
   pkt.randomize() with { addr == 8;};
   $display ("addr = %0d", pkt.addr);
end
```

- Soft constraint

A soft constraint is a constraint on a random variable, which allows overriding the constraint.

```
constraint addr_range {soft addr > 6;}
```

- Like class members, constraints also will get inherited from parent class to child class.
- Constraint blocks can be overridden by writing constraint block with the same name as in parent class.

7. What is Randomization

- Randomization is the process of generating random values to a variable.
- Verilog – Randomization of the variables alone ($random, $urandom).
- System Verilog –Randomization of variables and dynamic class objects(rand,randc,randomize()).
- rand

Variables declared with the rand keyword are standard random variables. Their values are uniformly distributed over their range.
Eg : rand bit [3:0] addr;

- randc

Variables declared with the randc keyword, on randomization variable values don't repeat a random value until every possible value has been assigned.
Eg : randc bit [3:0] addr;

- pre_randomize

The pre_randomize function can be used to set pre-conditions before the object randomization.

For example, Users can implement randomization control logic in pre_randomize function. i.e randomization enable or disable by using rand_mode() method.

• post_randomize

The post_randomization function can be used to check and perform post-conditions after the object randomization.

For example, Users can override the randomized values or can print the randomized values of variables.

• On calling randomize(),pre_randomize() and post_randomize() functions will get called before and after the randomize call respectively. Users can override the pre_randomize() and post_randomize() functions.

• rand_mode method

The rand_mode() method is used to disable the randomization of a variable declared with the rand/randc keyword.

```
module rand_methods;
        class packet;
           rand bit [2:0] addr1;
           randc bit [2:0] addr2;
     endclass
     initial
          begin
            packet pkt;
            pkt = new ();
            pkt.rand_mode(0); //disable rand_mode of object of class
            pkt. addr1.rand_mode (0); //disable rand_mode of variable of the class
            pkt.randomize(); //calling randomize method
      end
endmodule
```

8. What is Functional coverage

- Coverage is a way to measure the quality of testing/verification and completeness.
- Functional coverage is a user-defined metric that measures how much of the design specification has been exercised in verification.
- A user can use the coverage constructs to write coverpoints, cross coverage and bins and during simulation, the tools will monitor for coverage of these events.
- Functional coverage is more important in a random testing to measure the quality of stimulus and completeness.
- Functionality and design points are monitored using functional coverpoints and coverage tells how well those are covered.
- covergroup
  Address: coverpoint addr {

```
        bins low = {0,50};
        bins high = {51,100};
        }
endgroup
```

9. What are Assertions

- Assertions are used to validate the behavior of a design. Increases the observability and controllability of a design.
- Assertions provide an efficient way of improving overall design cycle productivity by cutting verification time.
- Assertion improves integration due to built-in self-checking, improves error detection and reduce debug time due to observability.
- There are two kinds of assertions: Immediate and Concurrent assertions
- Immediate assertions check for a condition at the current simulation time. An immediate assertion is the same as an if-else statement with assertion control. Immediate assertions have to be placed in a procedural block definition.
  `pass_statement; else fail_statement;`
- Concurrent assertions check the sequence of events spread over multiple clock cycles. The concurrent assertion is evaluated only at the occurrence of a clock tick. The test expression is evaluated at clock edges based on the sampled values of the variables involved. It can be placed in a procedural block, a module, an interface or a program definition. c_assert: assert property (@ (posedge clk) not (a && b));
- Eg: property delay_p checks for, signal “a” is asserted high on each clock cycle and if “a” is high in a cycle after two clock cycles, signal “b” has to be asserted high.

```
Property delay_p;

        @ (posedge clk) a ## 2 b;

endproperty

delay_check: assert property (delay_p);
```

10.Driver and sequence handshake

- In UVM, there is a mechanism to be followed when we want to send the transactions from the sequencer to the Driver in order to provide stimulus to the DUT.
- A particular Sequence is directed to run on a Sequencer which in turns further breaks down into a series of transaction items and these transaction items are needs to be transferred to the driver where these transaction items are converted into cycle-based signal/pin level transitions.

i)Sequencer Side Operation:

1. Creating the “transaction item” with the declared handle using factory mechanism.
2. Calling “start_item(<transaction_item_handle>)“. This call blocks the Sequencer till it grants the Sequence and transaction access to the Driver.
3. Randomizing the transaction OR randomizing the transaction with in-line constraints. Now the transaction is ready to be used by the Driver.
4. Calling “finish_item(<transaction_item_handle>)“. This call which is blocking in nature waits till Driver transfer the protocol related transaction data.

These are the operational steps from a Sequence which we want to execute using a Sequencer that is connected to a Driver inside an **“Agent”**.

ii)Driver Side Operation

1. Declaring the “transaction item” with a handle.
2. Calling the “get_next_item(<transaction_item_handle>)“. Default transaction_handle is “req”. “get_next_item()” blocks the processing until the “req” transaction object is available in the sequencer request FIFO & later “get_next_item” returns with the pointer of the “req” object.
3. Next, Driver completes its side protocol transfer while working with the virtual interface.
4. Calling the “item_done()” OR “item_done(rsp)“. It indicates to the sequencer the completion of the process. “item_done” is a non-blocking call & can be processed with an argument or without an argument. If a response is expected by the Sequencer/Sequence then item_done(rsp) is called. It results in Sequencer response FIFO is updatedwith the “rsp” object handle.

Sequence Code:

```
////////// Sequence Declaration

class my_seq extends uvm_sequence #(my_txn);

`uvm_object_utils(my_seq)

// Constructor

function new (string name = “”);

super.new(name);

endfunction: new

// Body Task

task body;

repeat(n)

begin
```

my_txn txn;

// Step 1

start_item(txn);

// Step 2

txn = my_txn::type_id::create (“txn”, this);

// Step 3

assert(txn.randomize() with (cmd == 0));

// Step 4

finish_item(txn);

end

endtask: body

endclass: my_seq

Following code is part of Driver code.

/////// Driver class class my_driver extends uvm_driver #(my_txn); `uvm_component_utils(my_driver) …… …… // Run Task task run_phase (uvm_phase phase); // Step 1 my_txn txn; forever begin // Step 2

seq_item_port.get_next_item(txn);

// Step 3

@ (posedge vif.clk)

begin

vif.addr = txn.addr;

vif.data = txn.data;

vif.we = txn.we;

end

// Step 4

seq_item_port.item_done();

end

endtask: run_phase

……

endclass: my_driver

11. Virtual sequencer

- A virtual sequencer is a sequencer that is not connected to a driver itself, but contains handles for sequencers in the testbench hierarchy. It is an optional component for running of virtual sequences - optional because they need no driver hookup, instead calling other sequences which run on real sequencers. If you only have a single driving
- agent, you do not need a virtual sequencer. If you have multiple driving agents you need a virtual sequencer.

12.Without configdb how to get interface in driver.

- SystemVerilog interface is static in nature, whereas classes are dynamic in nature. because of this reason, it is not allowed to declare the interface within classes, but it is allowed to refer to or point to the interface. A virtual interface is a variable of an interface type that is used in classes to provide access to the interface signals.

Virtual Interface declaration

//virtual interface

virtual intf vif;

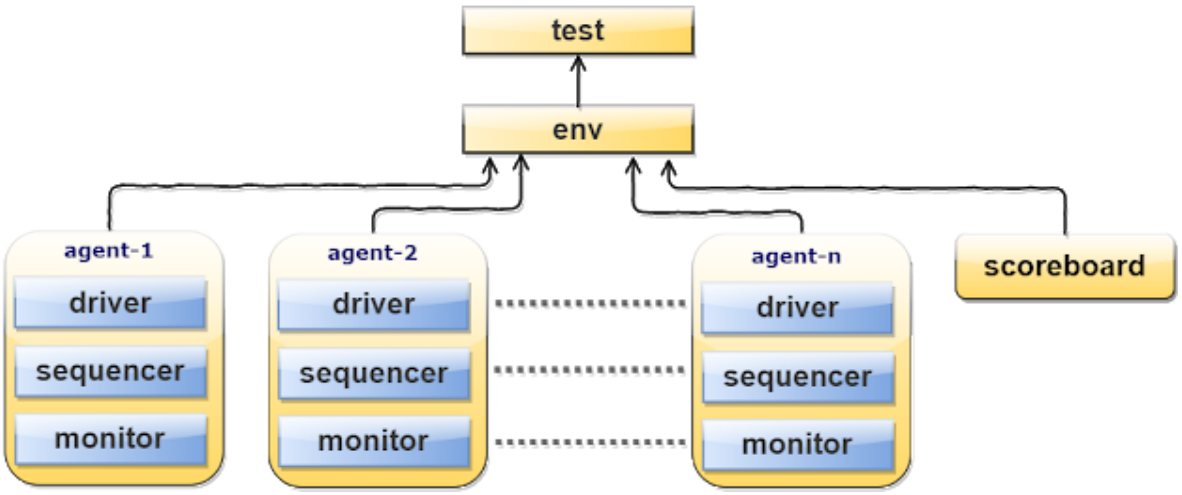
Connectingvirtualinterfacewith interface

```
//constructor
function new (virtual intf vif);
  //get the interface from test
  this.vif = vif;
endfunction
```

13.Fork join, how to execute threads sequentially in fork_join.

Fork-Join will start all the processes inside it parallel and wait for the completion of all the processes.Using “begin end” inside fork join we can execute threads sequentially.

14.From the top to agent tb flow



a) TOP

TestBench top is the module, it connects the DUT and Verification environment components. Typical Testbench_top contains

- DUT instance
- interface instance
- run_test() method
- virtual interface set config_db
  clock and reset generation logic

b)UVM test

The test is the topmost class. the test is responsible for,

- configuring the testbench.

- Initiate the testbench components construction process by building the next level down in the hierarchy ex: env.
- Initiate the stimulus by starting the sequence.

c)UVM Environment

Env or environment: The environment is a container component for grouping higher level components like agent's and scoreboard.

d)UVM Agent

UVM agent groups the uvm_components specific to an interface or protocol.
example: groups the components associated with BFM (Bus Functional Model).
The components of an agent are,

e) UVM SEQUENCE ITEM

The sequence-item defines the pin level activity generated by agent (to drive to DUT through the driver).

f) UVM DRIVER

Responsible for driving the packet level data inside sequence_item into pin level (to DUT).

g) UVM SEQUENCE

Defines the sequence in which the data items need to be generated and sent/received to/from the driver.

h) UVM SEQUENCER

Responsible for routing the data packet's(sequence_item) generated in sequence to the driver or vice-verse.

i)UVM MONITOR

Observes pin level activity on interface signals and converts into packet level which is sent to components such as scoreboards.

j) UVM Scoreboard

Receives data items from monitor's and compares with expected values. expected values can be either golden reference values or generated from the reference model.

15.Task and function

Tasks and Functions provide a means of splitting code into small parts.

A Task can contain a declaration of parameters, input arguments, output arguments, in-out arguments, registers, events, and zero or more behavioral statements.
SystemVerilog task can be,

- static
- automatic

i)Static tasks
Static tasks share the same storage space for all task calls.
ii)Automatic tasks

Automatic tasks allocate unique, stacked storage for each task call.

SystemVerilog allows,

- to declare an automatic variable in a static task
- to declare a static variable in an automatic task
- more capabilities for declaring task ports
- multiple statements within task without requiring a begin…end or fork…join block
- returning from the task before reaching the end of the task
- passing values by reference, value, names, and position
- default argument values
- the default direction of argument is input if no direction has been specified
- default arguments type is logic if no type has been specified

A Function can contain declarations of range, returned type, parameters, input arguments, registers, and events.

- A function without a range or return type declaration returns a one-bit value
- Any expression can be used as a function call argument
- Functions cannot contain any time-controlled statements, and they cannot enable tasks
- Functions can return only one value

SystemVerilog function can be,

- static
- automatic

i)Static Function

Static functions share the same storage space for all function calls.

ii)Automatic Function

Automatic functions allocate unique, stacked storage for each function call.

SystemVerilog allows,

- to declare an automatic variable in static functions
- to declare the static variable in automatic functions
- more capabilities for declaring function ports
- multiple statements within a function without requiring a begin…end or fork…join block
- returning from the function before reaching the end of the function
- Passing values by reference, value, names, and position
- default argument values
- function output and inout ports
- the default direction of argument is input if no direction has been specified. default arguments type is logic if no type has been specified.

16. Arguments to the task are static or dynamic?

A Task can contain a declaration of parameters, input arguments, output arguments, in-out arguments, registers, events, and zero or more behavioral statements.

Task can be,

- static
- automatic

i)Static tasks

Static tasks share the same storage space for all task calls.

ii)Automatic tasks

Automatic tasks allocate unique, stacked storage for each task call.

17. Asked to write uvm sequence, env, test code

- **UVM_sequence Code.**

```
class mem_sequence extends uvm_sequence#(mem_seq_item);

  `uvm_object_utils(mem_sequence)
```

```
    //Constructor
    function new (string name = "mem_sequence");
      super.new(name);
    endfunction
    virtual task body ();


      req = mem_seq_item::type_id::create("req"); //create the req (seq item)
      wait_for_grant();                          //wait for grant
      assert(req.randomize());                    //randomize the req
      send_request(req);                         //send req to driver
      wait_for_item_done();                       //wait for item done from driver
      get_response(rsp);                         //get response from driver

    endtask
  endclass
```

- **UVM_test code**

```
class mem_model_test extends uvm_test;

  `uvm_component_utils(mem_model_test)

  mem_model_env env;
  mem_sequence seq;
  function new (string name = "mem_model_test",uvm_component parent=null);
    super.new(name,parent);
  endfunction : new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = mem_model_env::type_id::create ("env", this);
    seq = mem_sequence::type_id::create("seq");
  endfunction : build_phase

  task run_phase(uvm_phase phase);
    seq.start(env.mem_agnt.sequencer);
  endtask : run_phase

endclass : mem_model_test
```

- **UVM Env code**

```
class mem_model_env extends uvm_env;

  mem_agent mem_agnt;
  `uvm_component_utils(mem_model_env)
  // new - constructor
  function new (string name, uvm_component parent);


    super.new(name, parent);
  endfunction : new
  // build_phase
  function void build_phase(uvm_phase phase);

    super.build_phase(phase);
    mem_agnt = mem_agent::type_id::create ("mem_agnt", this);
  endfunction : build_phase

endclass : mem_model_env
```

18.Clocking block and modports

A clocking block specifies timing and synchronization for a group of signals.

The clocking block specifies,

- The clock event that provides a synchronization reference for DUT and
- testbench
- The set of signals that will be sampled and driven by the testbench
  The timing, relative to the clock event, that the testbench uses to drive and sample those signals

Clocking block can be declared in interface, module or program block.

Clocking block terminologies

i)Clocking event

The event specification used to synchronize the clocking block, @ (posedge clk) is the clocking event.

ii)Clocking signal

Signals sampled and driven by the clocking block, from DUT and to DUT are the clocking signals,

iii)Clocking skew

Clocking skew specifies the moment (w.r.t clock edge) at which input and output clocking signals are to be sampled or driven respectively. A skew must be a constant expression and can be specified as a parameter.

iv)Input and Output skews

Input (or inout) signals are sampled at the designated clock event. If an input skew is specified then the signal is sampled at skew time units before the clock event. Similarly, output (or inout) signals are driven skew simulation time units after the corresponding clock event. A skew must be a constant expression and can be specified as a parameter. In case if the skew does not specify a time unit, the current time unit is used.

Modport

SystemVerilog Modport. The Modport groups and specifies the port directions to the wires/signals declared within the interface. modports are declared inside the interface with the keyword modport.

- By specifying the port directions, modport provides access restrictions
- The keyword modport indicates that the directions are declared as if inside the module
- Modport wire declared with input is not allowed to drive or assign, any attempt to drive leads to a compilation error
- The Interface can have any number of modports, the wire declared in the interface can be grouped in many modports
- Modpports can have, input, inout, output, and ref

19. RAL

- In a verification context, a register model (or register abstraction layer) is a set of classes that model the memory mapped behavior of registers and memories in the DUT in order to facilitate stimulus generation and functional checking (and optionally some aspects of functional coverage). The UVM provides a set of base classes that can be extended to implement comprehensive register modeling capabilities.
- The RAL (register abstraction layer) provides accesses to DUT and also keeps a track of register content of DUT.
- UVM RAL can be used to automate the creation of high-level, object-oriented abstraction model of registers and memory in DUT.
- Register layer makes the register abstraction and access of its contents independent of the bus protocol which is used to transfer data in and out of registers inside the design.
- Hierarchical model provided by RAL makes the reusability of test bench components very easy.

- The changes in initial configuration of registers or specifications can be easily communicated in the entire environment. RAL layer supports both front door and backdoor access. The backdoor access does not use the bus interface rather it uses the HDL defined paths for direct communication with the device. Thus, in zero simulation time the registers of device can be reconfigured using the backdoor access and verification can be started.
- One more advantage of backdoor access is that it can be used for verify if the access through front door is happening correctly. To achieve this the front door, write is verified using backdoor read.

20. How to get dut ouptut data into the uvm_sequence

- The output from DUT is received by the driver, from where it can be sent to sequence via sequencer. This can be done in two ways.
- By using item_done() driver sends an acknowledgement to sequencer that the output data has been received. And by using seq_item_port.item_done(rsp), a non-blocking method, driver sends the data to sequencer.
- By using another method(blocking), seq_item_port.put_response(rsp), driver sends data to sequencer and sequencer uses a method, get_response(rsp) to receive that data.

21. Write a packed array and unpacked array and initialise value and then assign packed array values to unpacked array?

```
module top;

      typedef logic[3:0]  packed_array;

      typedef logic unpacked_array[3:0];

      packed_array pa = 4'b1010;

      unpacked_array upa;

      initial

      begin

            upa = unpacked_array'(pa);

end

endmodule
```

22. What is polymorphism?

- Polymorphism allows a method to work in different ways depending on the handle of the class that calls that method ie., when there exists same method with different arguments or behavior, its functionality is decided by the handle of the class that called the method. There are two types of polymorphism, namely Static Polymorphism and Dynamic Polymorphism. Static polymorphism (or called as Compile-time polymorphism) is executed during the compile time. This type of polymorphism can be achieved by method

overloading or operator overloading which means changing the type of arguments or number of arguments of a method respectively.

23. write a coverage for memory [3:0]

0
1
2
3

Coverage for memory obtained by creating a coverpoint for the address and another coverpoint for read/write enable

25. where monitor connection will happen?

The connection between monitor and DUT happens in the connect phase of monitor through virtual interface.

The connection between monitor and scoreboard is established in the connect phase of environment, where the analysis port of monitor is connected to the analysis implementation port of scoreboard.

26. What is runtime polymorphism?

Run-time polymorphism, also known as Dynamic polymorphism, is that type of polymorphism in which call to the method is resolved during run-time. This can be achieved by Method overriding. Method overriding occurs when extended class has a method with same name as in base class but with different behavior. Polymorphism allows the extended class method to be accessed by parent class handle when that is method is declared virtual.